

Network Administration / System Administration

Homework 0

Student: 吳亞倫
ID: B13901011
Department: 電機工程學系

Network Administration

1. Short Answer (10pt)

2. Command Line Utilities (14pt)

註：在本題當中，操作系統環境如下：

- 作業系統：macOS Sequoia 15.2
- Command Line Tools: 16.2.0
- Shell: zsh 5.9 (arm64-apple-darwin24.0)

2.1 Trace Route

Reference:

- [Traceroute - Wikipedia](#) (Problem a)
- [What is Traceroute? - Varonis](#) (Problem a, d)
- [Traceroute Command in Linux with Examples - GeeksforGeeks](#) (Problem a, d)
- [Using traceroute to measure network latency and packet loss - Kadiska](#) (Problem a, d)
- [Linux Command to Translate Domain Name to IP - Stack Overflow](#) (Problem b)
- [Private Network - Wikipedia](#) (Problem c)
- [How to Read a Traceroute - Inmotion Hosting](#) (Problem d)
- [What does !Z and !X mean in a traceroute? - Server Fault](#) (Problem e)
- [Internet Control Message Protocol - Wikipedia](#) (Problem e)

Problem 2.1 (a) 我使用 traceroute 指令來追蹤封包的路徑，如以下所示：

```
1 $ traceroute speed.ntu.edu.tw
2 traceroute to speed.ntu.edu.tw (140.112.5.178), 64 hops max, 40 byte packets
3 1  10.200.200.200 (10.200.200.200)  8.739 ms  8.330 ms  8.086 ms
4 2  ip4-126.vpn.ntu.edu.tw (140.112.4.126)  6.536 ms  6.328 ms  6.970 ms
5 3  140.112.5.178 (140.112.5.178)  8.289 ms !Z  7.341 ms !Z  7.224 ms !Z
```

```

aaronwu@MacBookAir:~
> traceroute speed.ntu.edu.tw
traceroute to speed.ntu.edu.tw (140.112.5.178), 64 hops max, 40 byte packets
 1  10.200.200.200 (10.200.200.200)  8.739 ms  8.330 ms  8.086 ms
 2  ip4-126.vpn.ntu.edu.tw (140.112.4.126)  6.536 ms  6.328 ms  6.970 ms
 3  140.112.5.178 (140.112.5.178)  8.289 ms  !Z  7.341 ms  !Z  7.224 ms  !Z
>
~ >

```

Figure 1: Traceroute to speed.ntu.edu.tw

Problem 2.1 (b) 可以使用 ping、dig、host 或是 nslookup 指令，如 Figure 2 所示：

```

aaronwu@MacBookAir:~
> ping speed.ntu.edu.tw
PING speed.ntu.edu.tw (140.112.5.178): 56 data bytes
64 bytes from 140.112.5.178: icmp_seq=0 ttl=62 time=15.923 ms
64 bytes from 140.112.5.178: icmp_seq=1 ttl=62 time=12.611 ms
^C
--- speed.ntu.edu.tw ping statistics ---
2 packets transmitted, 2 packets received, 0.0% packet loss
round-trip min/avg/max/stddev = 12.611/14.267/15.923/1.656 ms
> nslookup speed.ntu.edu.tw
Server:      140.112.254.4
Address:     140.112.254.4#53

Name:   speed.ntu.edu.tw
Address: 140.112.5.178
> dig +short speed.ntu.edu.tw
140.112.5.178
> host speed.ntu.edu.tw
speed.ntu.edu.tw has address 140.112.5.178
~ >

```

Figure 2: Domain to IP Address Commands

Problem 2.1 (c) 在 (a) 小題當中，總共解析出三個 IP 位址，分別是：

- 10.200.200.200 (Private Network)
- 140.112.4.126 (Public Network)
- 140.112.5.178 (Public Network)

其中，根據 IPv4 的私有 IP 定義 (RFC 1918)，私有 IP 地址範圍如下：

- Class A: 10.0.0.0 -- 10.255.255.255
- Class B: 172.16.0.0 -- 172.31.255.255
- Class C: 192.168.0.0 -- 192.168.255.255

因此，10.200.200.200 是一個 Private Network 的 IP 位址。

另外，若我在瀏覽器當中輸入 10.200.200.200，則會出現台大 sslvpn 登入頁面的畫面（如 Figure 3 所示），但若關閉 VPN 連線後，則無法連線至該 IP 位址。有此可以推知該 IP 位址為位於台大內網的 sslvpn 的伺服器 IP 位址，屬於 Private Network。



Figure 3: 10.200.200.200 連線後頁面

至於另外兩個 IP 位址，皆不在私有 IP 地址範圍內，因此可以認為公有 IP 地址。

Problem 2.1 (d) 在 traceroute 的回傳結果當中 (Figure 1)，同一列的三個時間代表的是針對網路當中每一個節點進行三次測試所得到的結果 (3 tests per TTL value)。透過多次的測量，可以檢查網路的穩定性，以及路徑是否具有一制性。而在不同列當中，越底下的時間並不一定比較大。因為每一個節點處理這種封包的 Priority 不同，可能會因為網路負載的不同而有所差異。因此，TTL 值越大，所需的時間不一定會越長。

Problem 2.1 (e) 以下為 traceroute 的路徑圖 (Figure 4)：

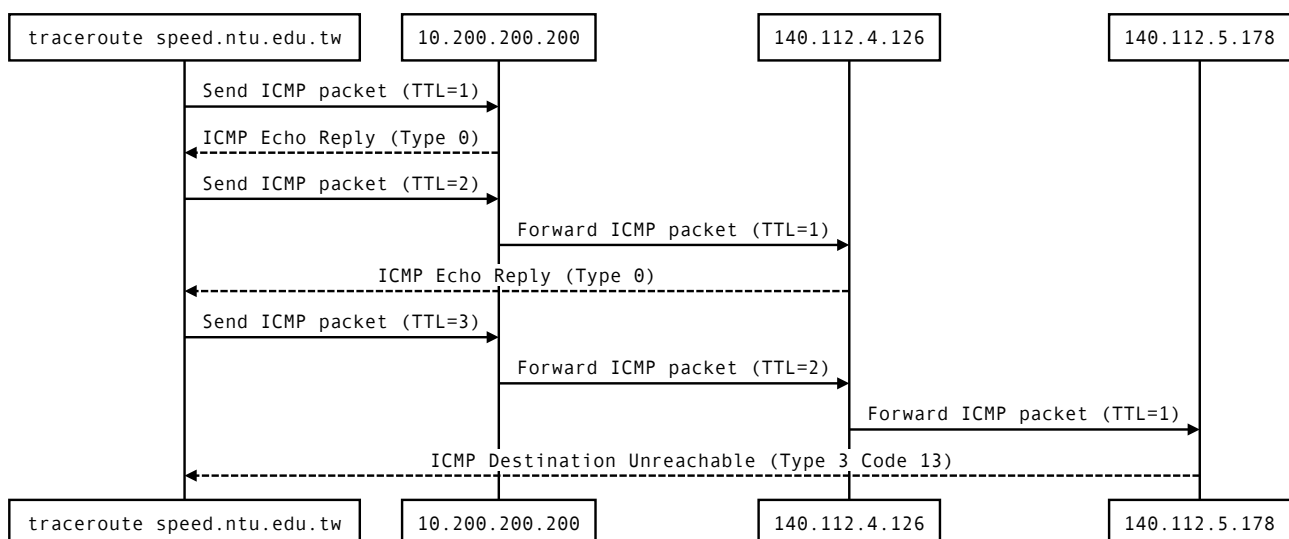


Figure 4: Traceroute Sequence Graph

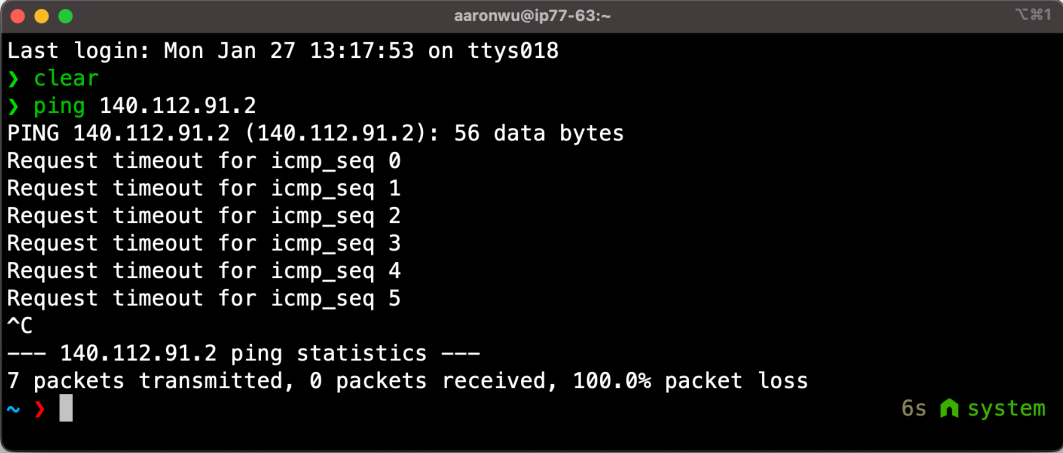
2.2 伺服器 140.112.91.2

Reference:

- [Ping \(networking utility\) - Wikipedia](#) (Problem a)
- [ICMP 協定是什麼? 如何應用 Ping 命令 - iThome](#) (Problem a)

Problem 2.2 (a) ping

1. 在使用 ping 指令時，發送的訊息為 ICMP 的 echo-request 封包，而接收的訊息為 ICMP 的 echo-reply 封包。當伺服器無法回應時，則會出現 Request timeout 的訊息。
2. 下圖 (Figure 5) 為指令 ping 140.112.91.2 的回傳。



```
aaronwu@ip77-63:~  
Last login: Mon Jan 27 13:17:53 on ttys018  
> clear  
> ping 140.112.91.2  
PING 140.112.91.2 (140.112.91.2): 56 data bytes  
Request timeout for icmp_seq 0  
Request timeout for icmp_seq 1  
Request timeout for icmp_seq 2  
Request timeout for icmp_seq 3  
Request timeout for icmp_seq 4  
Request timeout for icmp_seq 5  
^C  
--- 140.112.91.2 ping statistics ---  
7 packets transmitted, 0 packets received, 100.0% packet loss  
~ >
```

Figure 5: Screenshot of ping 140.112.91.2

Problem 2.2 (b) nmap

3. Basic Wireshark (11pt)
4. Cryptography (5pt)
5. 為什麼簽不了憑證??? (10pt)

System Administration

6. btw I use arch (15pt)

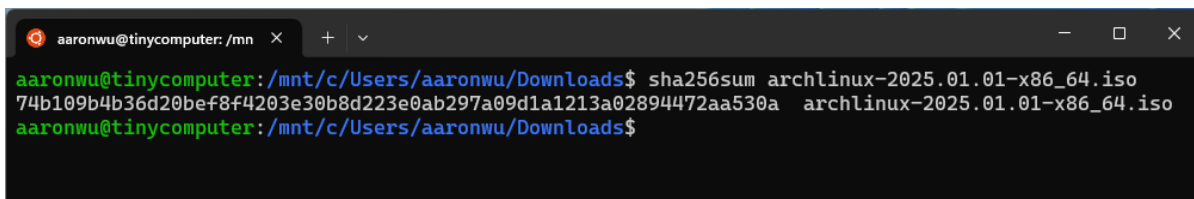
Reference:

- [Arch Linux Installation Guide \(正體中文\)- ArchWiki](#)
- [VirtualBox/Install Arch Linux as a guest - ArchWiki](#)
- [How to Dual Boot Arch Linux and Windows 11 \(2025\) - Ksk Royal \(youtube\)](#)
- [How to Dual Boot Arch Linux and Windows 11 \(2024\) - Ksk Royal \(youtube\)](#)
- [Swap - ArchWiki](#)
- [Linux Find Out My Machine Name/Hostname - nixCraft](#)
- 討論對象：chatGPT、喻慈恩

Problem 6.0 安裝流程

我使用虛擬機器 VirtualBox 來安裝 Arch Linux，操作環境為 Windows 11，安裝過程如下：

1. 下載 Arch Linux 的 ISO 檔案，並且在 wsl (ubuntu) 環境下使用 sha256sum 檢查 iso 檔案的 hash 值。



```
aaronwu@tinycomputer: /mnt/c/Users/aaronwu/Downloads$ sha256sum archlinux-2025.01.01-x86_64.iso
74b109b4b36d20bef8f4203e30b8d223e0ab297a09d1a1213a02894472aa530a archlinux-2025.01.01-x86_64.iso
aaronwu@tinycomputer: /mnt/c/Users/aaronwu/Downloads$
```

Figure 6: Screenshot of sha256sum

2. 下載並安裝 VirtualBox
3. 建立一個新的虛擬機器，配置如下：
 - 名稱：archlinux
 - ISO 映像：選擇剛剛下載的 Arch Linux 的 ISO 檔案
 - 類型：Linux
 - 版本：Arch Linux (64-bit)
 - 記憶體大小：2 GB
 - 處理器：2 CPUs
 - 啟用 EFI
 - 硬碟：建立新的硬碟，大小 32 GB

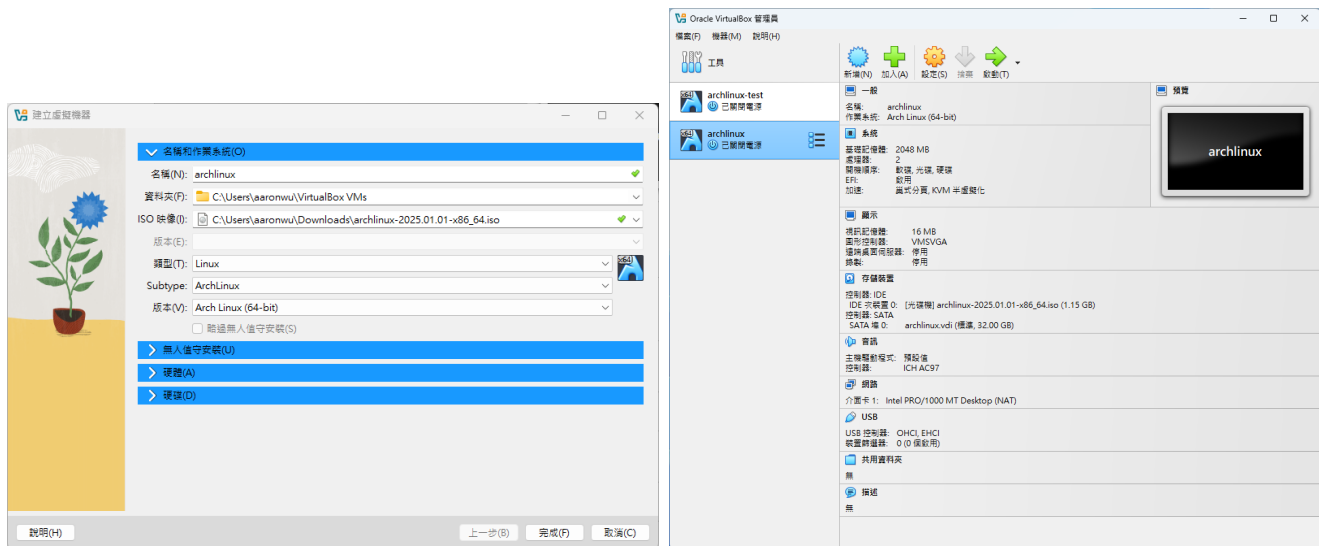


Figure 7: VM Configuration

4. 啟動虛擬機器，選擇 Arch Linux install medium (x86_64, UEFI) 並按下 Enter 進入安裝 Live 環境。

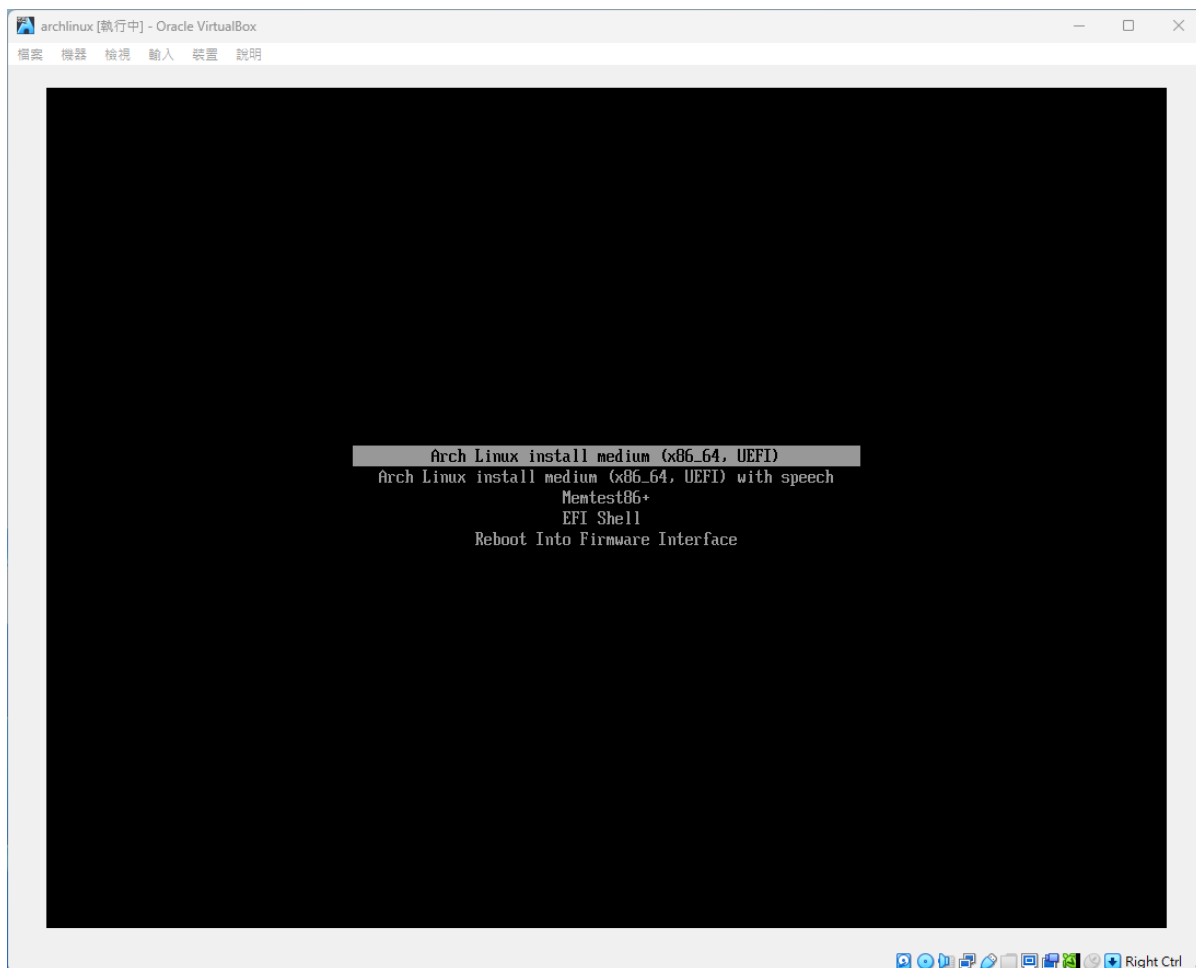


Figure 8: 進入 Live 環境

5. 調整字體大小：`setfont ter-122n`
6. 確認網路連線：`ping -c 5 google.com`

```
root@archiso ~ # ping -c 5 google.com
PING google.com (142.250.196.206) 56(84) bytes of data:
64 bytes from nctsaa-ac-in-f14.1e100.net (142.250.196.206): icmp_seq=1 ttl=255 time=8.34 ms
64 bytes from nctsaa-ac-in-f14.1e100.net (142.250.196.206): icmp_seq=2 ttl=255 time=5.42 ms
64 bytes from nctsaa-ac-in-f14.1e100.net (142.250.196.206): icmp_seq=3 ttl=255 time=5.76 ms
64 bytes from nctsaa-ac-in-f14.1e100.net (142.250.196.206): icmp_seq=4 ttl=255 time=6.02 ms
64 bytes from nctsaa-ac-in-f14.1e100.net (142.250.196.206): icmp_seq=5 ttl=255 time=5.18 ms

--- google.com ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4013ms
rtt min/avg/max/mdev = 5.175/6.142/8.343/1.137 ms
root@archiso ~ #
```

Figure 9: 確認網路連線

7. 確認系統時間同部：`timedatectl`
8. 安裝必要套件，執行以下指令更新 Pacman 並安裝套件：

```
1 pacman -Sy
2 pacman -S archlinux-keyring
3 pacman -S archinstall
```

9. 建立硬碟分割表，並建立分割區：

(a) 使用 `lsblk` 查看目前磁碟分割情況

```
root@archiso ~ # lsblk
NAME MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
loop0  7:0    0 820.6M  1 loop /run/archiso/airootfs
sda     8:0    0   32G   0 disk
sr0     11:0   1    1.1G   0 rom  /run/archiso/bootmnt
root@archiso ~ #
```

Figure 10: 查看目前磁碟分割情況: `/dev/sda` 為主要硬碟區

(b) 使用 `cfdisk` 進行分割，並建立以下分割區：

Partition	Type	Size	Mount Point
<code>/dev/sda1</code>	EFI System	1G	<code>/boot</code>
<code>/dev/sda2</code>	Linux filesystem	15G	<code>/</code>
<code>/dev/sda3</code>	Linux filesystem	5G	<code>/home</code>
<code>/dev/sda4</code>	Linux swap	2G	<code>swap</code>

詳細步驟如下：

- i. `cfdisk /dev/sda`
- ii. 選擇 `gpt` 作為分割表格式
- iii. 建立 `/dev/sda1`：移動到 Free Space → 按 New → 按 Enter → 輸入 1G → 再按 Enter → 選擇 Type → 設為 EFI System。
- iv. 建立 `/dev/sda2`：移動到 Free Space → 按 New → 按 Enter → 輸入 15G → 再按 Enter → 選擇 Type → 設為 Linux filesystem。
- v. 建立 `/dev/sda3`：移動到 Free Space → 按 New → 按 Enter → 輸入 5G → 再按 Enter → 選擇 Type → 設為 Linux filesystem。
- vi. 建立 `/dev/sda4`：移動到 Free Space → 按 New → 按 Enter → 輸入 2G → 再按 Enter → 選擇 Type → 設為 Linux swap。

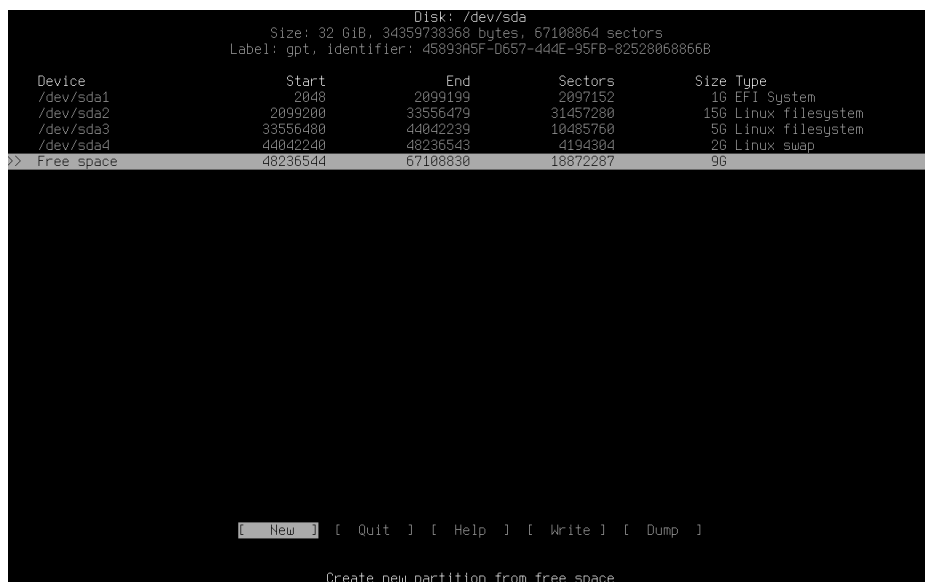


Figure 11: cfdisk 分割區設定

- vii. 按 Write 並確認，再按 Quit 離開。

(c) 使用 `lsblk` 確認分割區是否正確

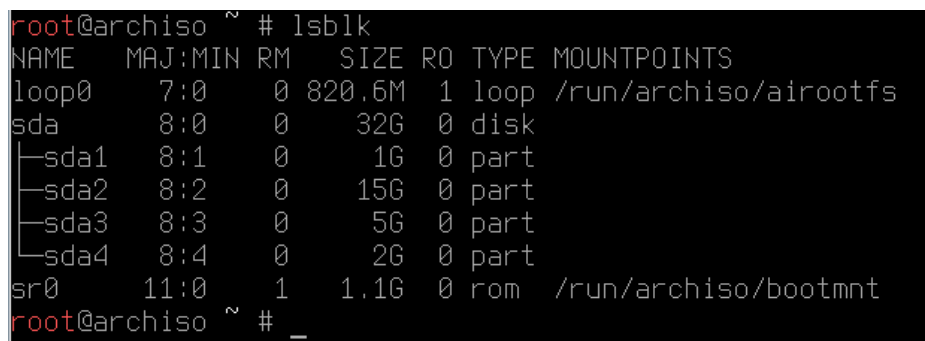


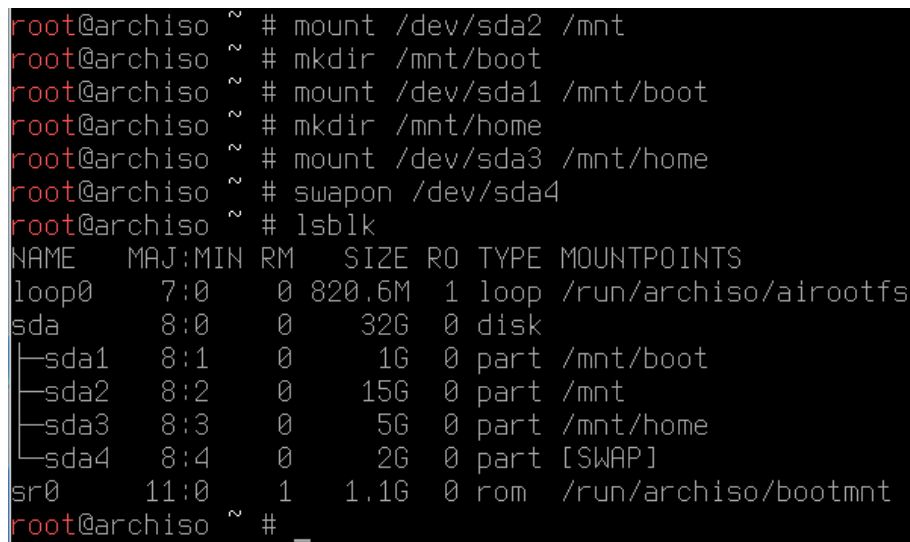
Figure 12: 確認分割區是否正確

10. 格式化分割區：

```
1 mkfs.fat -F32 /dev/sda1
2 mkfs.ext4 /dev/sda2
3 mkfs.ext4 /dev/sda3
4 mkswap /dev/sda4
```

11. 掛載分割區：

```
1 mount /dev/sda2 /mnt
2 mkdir /mnt/boot
3 mount /dev/sda1 /mnt/boot
4 mkdir /mnt/home
5 mount /dev/sda3 /mnt/home
6 swapon /dev/sda4
```



```
root@archiso ~ # mount /dev/sda2 /mnt
root@archiso ~ # mkdir /mnt/boot
root@archiso ~ # mount /dev/sda1 /mnt/boot
root@archiso ~ # mkdir /mnt/home
root@archiso ~ # mount /dev/sda3 /mnt/home
root@archiso ~ # swapon /dev/sda4
root@archiso ~ # lsblk
NAME        MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
loop0       7:0    0 820.6M 1 loop /run/archiso/airootfs
sda         8:0    0   32G  0 disk
├─sda1      8:1    0    1G  0 part /mnt/boot
├─sda2      8:2    0   15G  0 part /mnt
├─sda3      8:3    0    5G  0 part /mnt/home
└─sda4      8:4    0    2G  0 part [SWAP]
sr0        11:0    1   1.1G  0 rom  /run/archiso/bootmnt
root@archiso ~ # _
```

Figure 13: 掛載分割區

12. 安裝基本系統：輸入 archinstall，各個選項如下：

- Archinstall language: English (100%) (預設)
- Locales:
 - Keyboard layout: us (預設)
 - Locale language: en_US (預設)
 - Locale encoding: UTF-8 (預設)

- Mirrors: 選擇臺灣或其他東亞鄰境國家
- Disk configuration:
 - Partitioning: Pre-mounted configuration
 - Root mount directory: /mnt

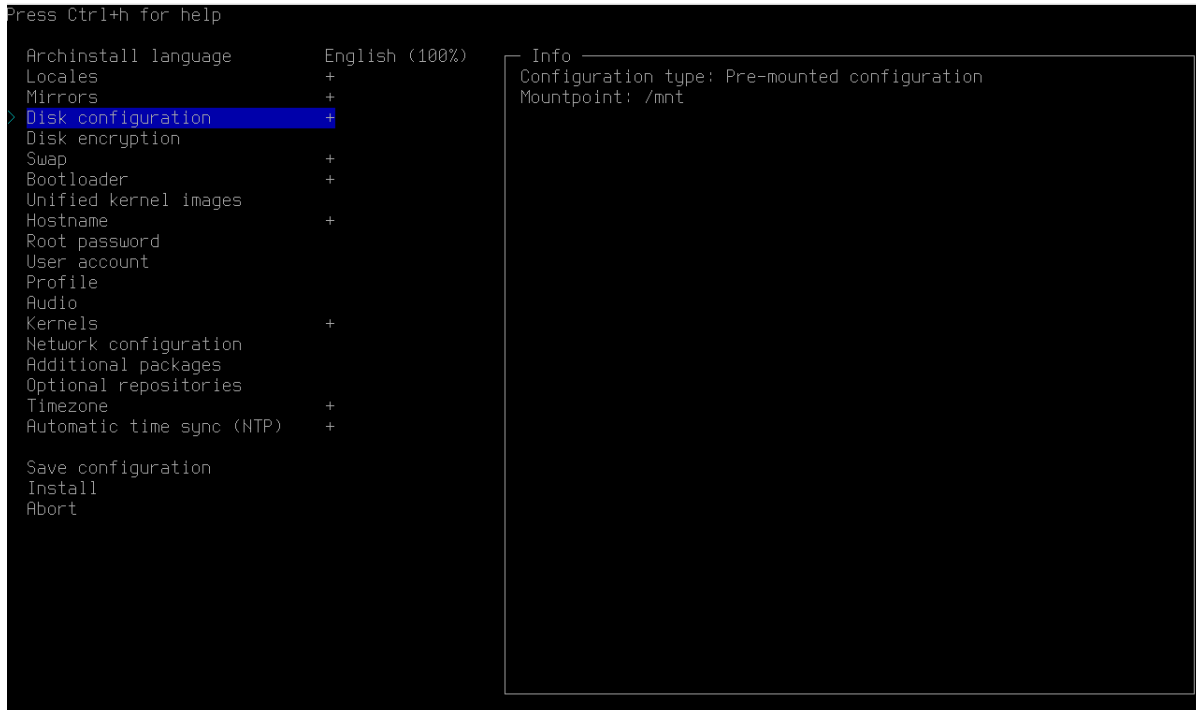


Figure 14: Disk Configuration

- Swap: 可以開啟 swap on zram，這樣除了我們配置的 swap 分割區外，還會開啟一個 swap on zram，可以提高系統的效能。
- Bootloader: 選擇 Grub
- Hostname: 學號
- Root password、User account: 設定 root 密碼、新增使用者
- Profile: 選擇 Server，我選擇裡面的 sshd
- Network configuration: 選擇 NetworkManager
- Timezone: Asia/Taipei
- Automatic time sync (NTP): enable

13. 選擇完成後，按下 Install 開始安裝 (Figure 15)

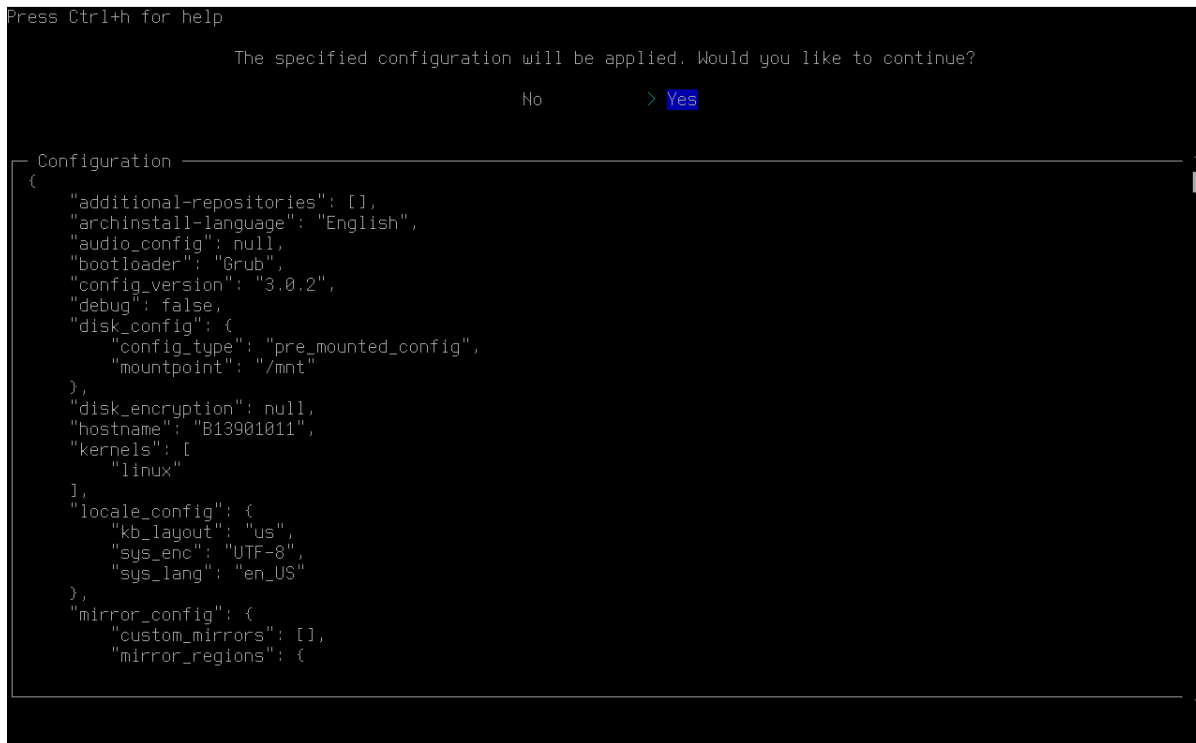


Figure 15: 按下 Install 之後，確認各項設定，之後點選 Yes 進行安裝

14. 安裝完成後，選擇 yes，來 chroot 進入新安裝的系統

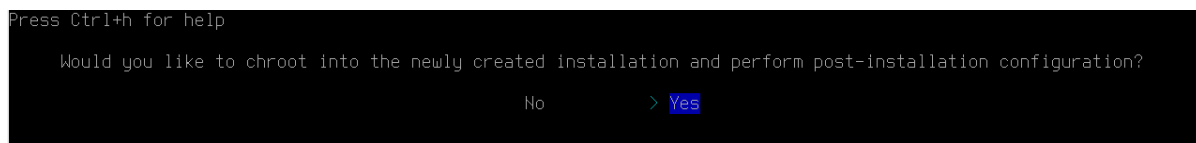


Figure 16: chroot 進入新安裝的系統

15. 安裝必要套件 sudo：

```

1 pacman -Syu
2 pacman -S wget curl git vim gcc neofetch htop

```

16. 安裝 Grub：Archinstall 所安裝的 Grub 有時會有問題，因此我們需要重新安裝 Grub：

```

1 pacman -S grub efibootmgr dosfstools mtools
2 grub-install --target=x86_64-efi --efi-directory=/boot --bootloader-id=GRUB
3 grub-mkconfig -o /boot/grub/grub.cfg

```

17. 輸入 exit 離開 chroot

18. 輸入 `shutdown now` 關機，並移除 iso 檔案，重新啟動虛擬機器

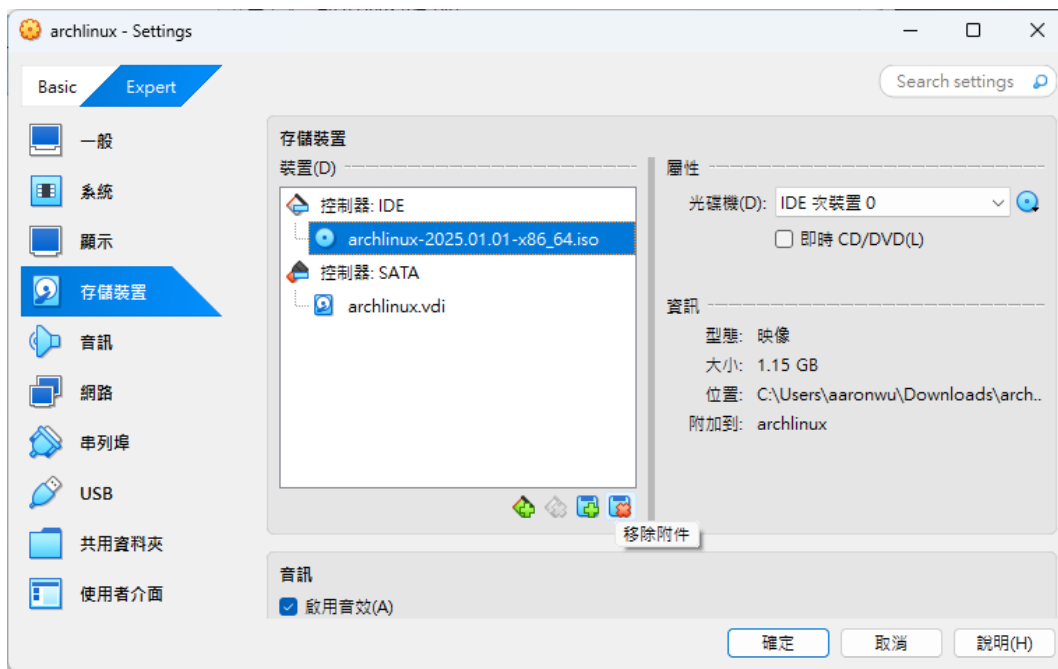


Figure 17: 移除 iso 檔案，重新啟動虛擬機器

19. 開機，使用 Grub 選擇 Arch Linux 進入系統

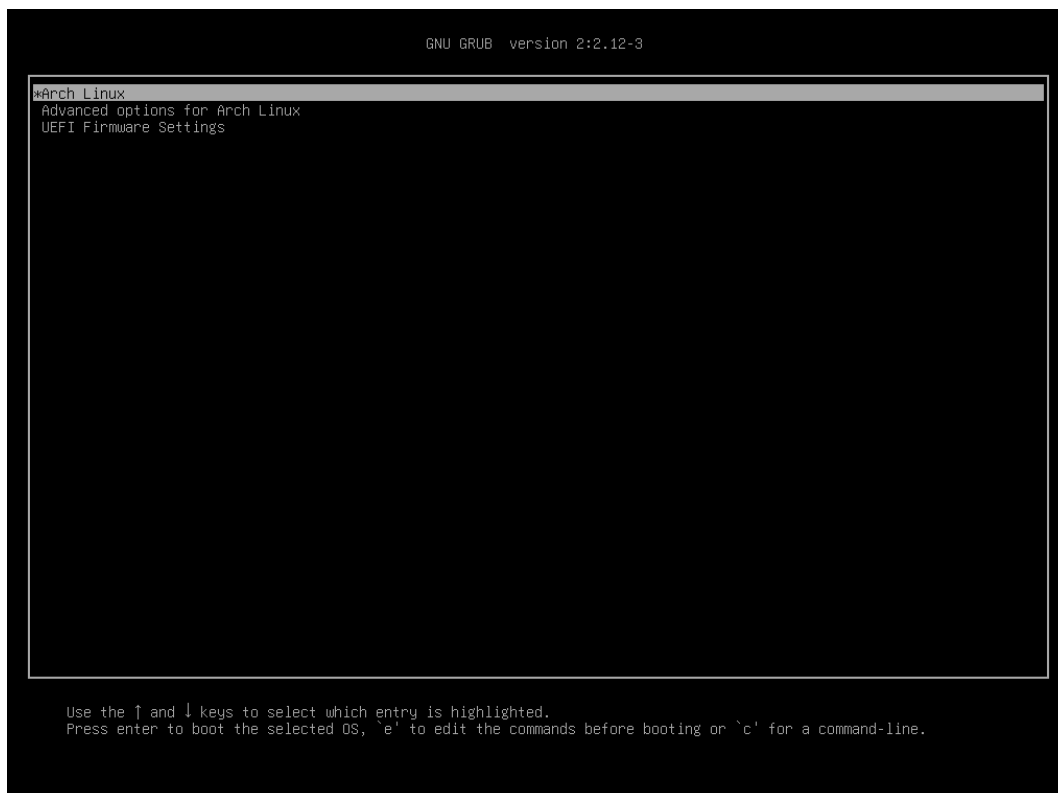


Figure 18: 使用 Grub 選擇 Arch Linux 進入系統

20. 輸入 `nasa` / `nasa` 登入
21. 確認網路連線: `ping -c 3 google.com`
22. 完成安裝。

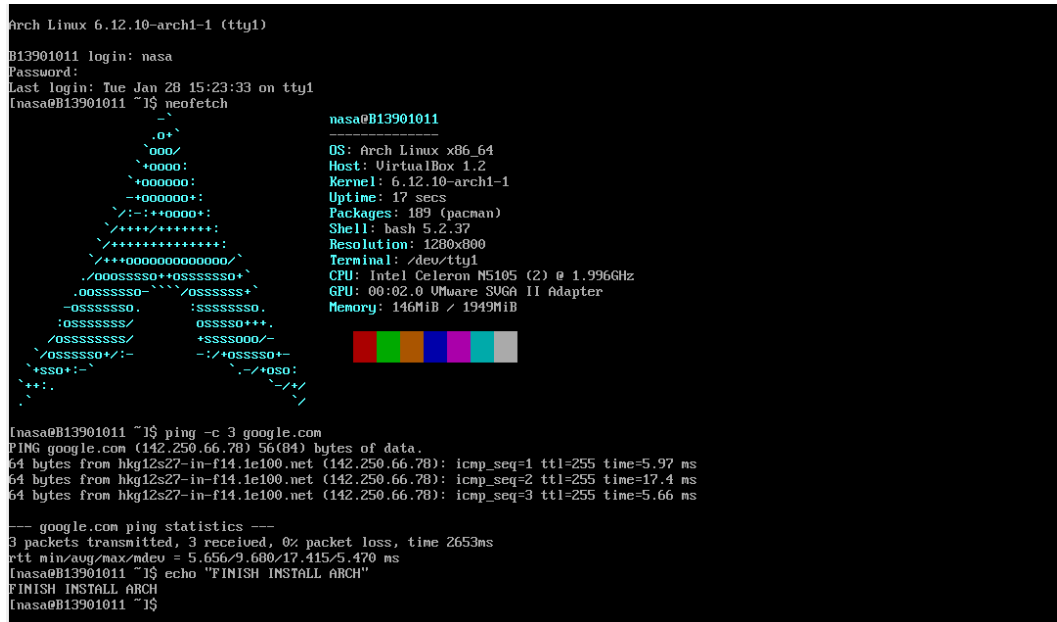


Figure 19: 完成安裝

Problem 6.1 host name

- 在使用 `archinstall` 安裝 Arch Linux 時，我已經將主機名稱設定為學號 B13901011
- 可使用以下指令更改主機名稱: `sudo hostnamectl set-hostname <hostname>`
- 可以使用以下指令查看主機名稱: `sudo hostnamectl`

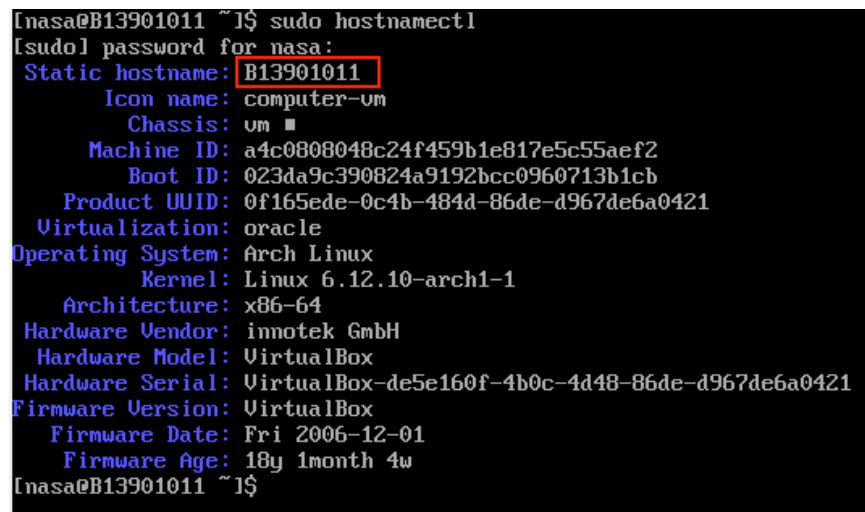
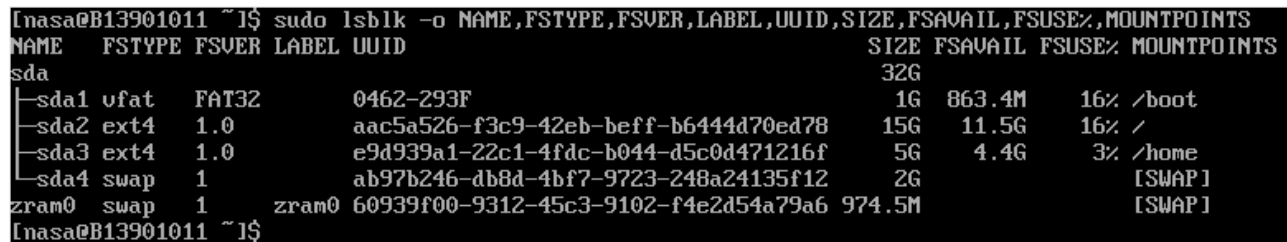


Figure 20: Screenshot of `hostnamectl`

Problem 6.2 uuid 與磁碟空間分配

- 使用指令 `lsblk` 搭配 `-o` 選項可以查看磁碟分割區以及個字的 UUID

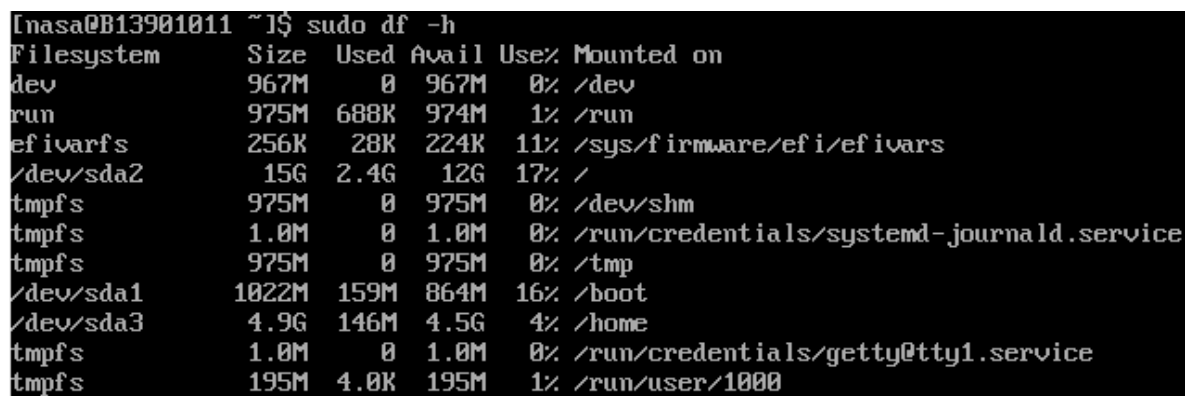
```
1 sudo lsblk -o NAME,FSTYPE,FSVER,LABEL,UUID,SIZE,FSAVAIL,FSUSE%,MOUNTPOINT
```



```
[nasa@B13901011 ~]$ sudo lsblk -o NAME,FSTYPE,FSVER,LABEL,UUID,SIZE,FSAVAIL,FSUSE%,MOUNTPOINTS
NAME FSTYPE FSVER LABEL UUID                               SIZE FSAVAIL FSUSE% MOUNTPOINTS
sda                                     32G
├─sda1 vfat    FAT32    0462-293F                               1G  863.4M    16% /boot
├─sda2 ext4    1.0      aac5a526-f3c9-42eb-beff-b6444d70ed78 15G  11.5G    16% /
├─sda3 ext4    1.0      e9d939a1-22c1-4fdc-b044-d5c0d471216f  5G   4.4G     3% /home
├─sda4 swap    1        ab97b246-db8d-4bf7-9723-248a24135f12  2G
zram0 swap    1        60939f00-9312-45c3-9102-f4e2d54a79a6 974.5M
[SWAP]
```

Figure 21: Screenshot of `lsblk`

- 另外，使用指令 `df -h` 也可以查看磁碟的空間分配



```
[nasa@B13901011 ~]$ sudo df -h
Filesystem      Size  Used Avail Use% Mounted on
dev              967M   0    967M   0% /dev
run             975M  688K  974M   1% /run
efivarfs        256K   28K  224K  11% /sys/firmware/efi/efivars
/dev/sda2        15G   2.4G   12G  17% /
tmpfs            975M   0    975M   0% /dev/shm
tmpfs            1.0M   0    1.0M   0% /run/credentials/systemd-journald.service
tmpfs            975M   0    975M   0% /tmp
/dev/sda1       1022M  159M  864M  16% /boot
/dev/sda3        4.9G  146M  4.5G   4% /home
tmpfs            1.0M   0    1.0M   0% /run/credentials/getty@tty1.service
tmpfs            195M   4.0K  195M   1% /run/user/1000
```

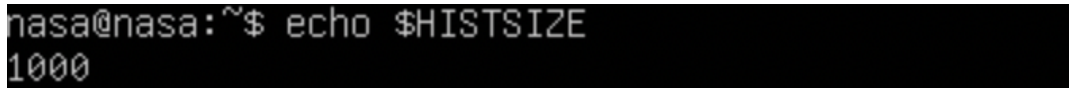
Figure 22: Screenshot of `sudo df -h`

Problem 6.3 Linux Distribution 與 kernel 版本

- 使用指令 `neofetch` 可以查看 Linux Distribution 與 kernel 版本 (Figure 23)
- 另外，也可以使用 `sudo cat /etc/os-release` 查看 Distribution
- 或是使用 `uname -r` 查看 kernel 版本

Problem 7.1 (b) bash session 可暫存指令數

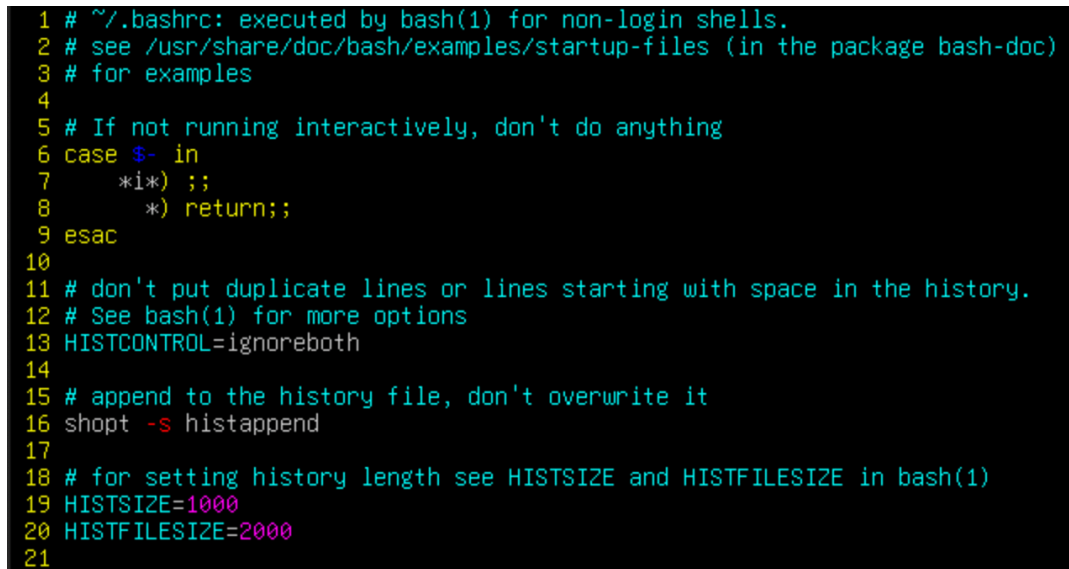
- 可以查看環境變數 HISTSIZE 來查看 bash session 可暫存指令數的上限，結果如下：



```
nasa@nasa:~$ echo $HISTSIZE
1000
```

Figure 25: Screenshot of echo \$HISTSIZE

- 一般來說，環境變數的設定都是記載在 .bashrc 檔案中，查看檔案後發現第 19 行的確有 HISTSIZE 的設定。



```
1 # ~/.bashrc: executed by bash(1) for non-login shells.
2 # see /usr/share/doc/bash/examples/startup-files (in the package bash-doc)
3 # for examples
4
5 # If not running interactively, don't do anything
6 case $- in
7     *i*) ;;
8     *) return;;
9 esac
10
11 # don't put duplicate lines or lines starting with space in the history.
12 # See bash(1) for more options
13 HISTCONTROL=ignoreboth
14
15 # append to the history file, don't overwrite it
16 shopt -s histappend
17
18 # for setting history length see HISTSIZE and HISTFILESIZE in bash(1)
19 HISTSIZE=1000
20 HISTFILESIZE=2000
21
```

Figure 26: .bashrc，可以看到第 19 行宣告了環境變數 HISTSIZE 的值

- 若修改 .bashrc 當中 HISTSIZE 的值，再重新開啟一個 bash session，可以發現 HISTSIZE 的值已經被修改。因此，可以確認 HISTSIZE 的確是記錄在 /home/nasa/.bashrc 檔案中。

Problem 7.1 (c) history file 歷史指令數上限

- 可以查看環境變數 HISTFILESIZE 來查看 history file 歷史指令數的上限，結果如下：



```
nasa@nasa:~$ echo $HISTFILESIZE
2000
nasa@nasa:~$
```

Figure 27: Screenshot of echo \$HISTFILESIZE

- 同上題，可以在 `.bashrc` 的第 20 行找到 `HISTFILESIZE` 的設定。

```

15 # append to the history file, don't overwrite it
16 shopt -s histappend
17
18 # for setting history length see HISTSIZE and HISTFILESIZE in bash(1)
19 HISTSIZE=1000
20 HISTFILESIZE=2000

```

Figure 28: `.bashrc`，可以看到第 20 行宣告了環境變數 `HISTFILESIZE` 的值

- 因此，可以確認 `HISTFILESIZE` 的確是記錄在 `/home/nasa/.bashrc` 檔案中。

Problem 7.1 (d) flag

- 首先，查看預設的 history file (`/home/nasa/.bash_history`) 的內容，發現裡面有記載將 flag 輸出到新的 history file 第 104 行的內容：

```

./gen_flag --line 104 --out new_history_file
exit
echo $HISTSIZE
exit
~

```

Figure 29: Screenshot of `/.bash_history`

- 結合 (a) 小題的內容，前往 `/kickstart.nvim/.git/logs/refs/remotes/origin/HEAD` 查看第 104 行的內容，即可找到 flag。
- 我使用 `vim` 來查看該檔案，並設定開啟行號 (`:set nu`)，跳到第 104 行找到內容如下：

```

100 echo NASA{iIaz04Jfm1BC8zsCWuZV}
101 echo NASA{w6RVZvXg4uqgibghZMhh}
102 echo NASA{IyVQCtV49x0bMpTi2oJL}
103 echo NASA{7iPKzA0jfJFbBylP0tMp}
104 echo NASA{y0UF1nd+heCoRr3tFL4G}
105 echo NASA{YQCTko59IYctjaCAH0aZ}
106 echo NASA{98m1di852BmbLAqINTjB}
107 echo NASA{4LR2KXamAXwVIncDiLsa}
108 echo NASA{Pu2509fM9142uma30NyX}
109 echo NASA{VlrzyU2GDV12lgCqph2v}
:104

```

Figure 30: Find the flag in new history file

Flag of Problem 7.1 (d):

`NASA{y0UF1nd+heCoRr3tFL4G}`

7.2 寶藏箱 (/home/nasa/treasure)(5pt)

Reference:

- <https://www.cyberciti.biz/faq/linux-ls-command-sort-by-file-size/>

- 首先，執行 /home/nasa/treasure，得到以下結果

```
nasa@nasa:~$ ./treasure
Searching for treasures....(It might take some time)
You found a really big treasure chest!
But there are many garbage around...
What you are looking for is in the smallest file, and at the start of line 134.
That's all I know for now...
Good luck!
nasa@nasa:~$
```

Figure 31: /home/nasa/treasure 執行結果

- 根據要求進到目錄中，找到大量 flag-xxx 的檔案。我使用以下指令使 ls -l 指令依據檔案大小排序 (-S)，並存到 temp.txt 當中，方便檢視。

```
1 ls -lS > temp.txt
```

- 在 vim 當中檢視 temp.txt，可以看到 flag-xxx 檔案的大小依序遞減，並找到最小的檔案 flag-109。

```
-rw-rw-r-- 1 nasa nasa 2016014 Jan 29 14:22 flag-407
-rw-rw-r-- 1 nasa nasa 2015007 Jan 29 14:22 flag-468
-rw-rw-r-- 1 nasa nasa 2015007 Jan 29 14:24 flag-919
-rw-rw-r-- 1 nasa nasa 1879062 Jan 29 14:21 flag-109
-rw-rw-r-- 1 nasa nasa      0 Jan 29 14:26 temp.txt
"temp.txt" 1002L, 53067B
```

Figure 32: 檢視 temp.txt，找到最小的檔案 flag-109

- 最後，查看 flag-109 的第 134 行，即可找到 flag。

```
134 NASA{EZ_TrEa$Ur3_HunT!}_6ibiRlwM28vHyfDofql8RRVYBksoVHrV3w5QUpM9eIMQ5
wfJ9KEOrRtfk26{KuGGotiY0hgQ0o9M6sl97lLt21MwlMr2v7JHw1ouuOudaaEaG1kZoc
}KIV7OQ7agpB5YLMXC07UC}xt{jCLbkGUzf}3b6rep5NrLS_tM4xv3io9Z1USwCGHPHD8
QqSjbi0ks1lFrENuHdzQtCdfCgekQKktCMX{yEgNl811CoKI9iuh_HJN9v9T}fsrY7ed
```

Figure 33: 查看 flag-109 的內容

Flag of Problem 7.2:

```
NASA{EZ_TrEa$Ur3_HunT!}
```

7.3 魔物 (/home/nasa/boss)(5pt)

Reference:

- <https://stackoverflow.com/questions/13671750/what-does-mean-or-in-bash>
- <https://www.runoob.com/linux/linux-comm-pkill.html>

- 首先，執行 /home/nasa/boss，得到以下結果

```
nasa@nasa:~$ ./boss
If you can kill all my subprocesses within 3 seconds, I will show you my secret!
```

Figure 34: /home/nasa/boss 執行結果

- 我使用以下指令成功在三秒內 kill 掉 boss 的所有 subprocess，得到 flag。

```
1 ./boss & pkill -P $!
```

- 這裡 \$! 代表最後一個背景執行的程序的 PID，因此 pkill -P \$! 會 kill 掉所有 boss 的 subprocess。

```
nasa@nasa:~$ ./boss & pkill -P $!
[2] 1526
If you can kill all my subprocesses within 3 seconds, I will show you my secret!
Well done!
NASA{m0dERn_Pr0B1em$_reQU1r3_m0dERn_S0luT10N5}
[2]+  Done                  ./boss
nasa@nasa:~$ _
```

Figure 35: 得到 flag

Flag of Problem 7.3:

```
NASA{m0dERn_Pr0B1em$_reQU1r3_m0dERn_S0luT10N5}
```

7.4 通關密語 (/home/nasa/chal)(5pt)

Reference:

- <https://eecsmt.com/linux/linux-strings/>
- <https://blog.gtwang.org/linux/linux-grep-command-tutorial-examples/>
- [How to pass command output as multiple arguments to another command - Stack Overflow](#)

- 首先，執行 /home/nasa/chal，得到以下結果

```
nasa@nasa:~$ ./chal
Usage: ./chal <passcode>
nasa@nasa:~$ _
```

Figure 36: /home/nasa/chal 執行結果

- 根據題目提示，我們可以使用 strings 指令來將二進位檔 chal 的可閱讀部份輸出，並使用 grep 找到含有特定字串的通關密語。

```
1 strings chal | grep 486 | grep re02
```

- 最後，利用 \$(...) 的語法，將 strings chal | grep 486 | grep re02 的輸出作為參數傳入 ./chal，即可得到 flag。

```
1 ./chal $(strings chal | grep 486 | grep re02)
```

```
nasa@nasa:~$ strings chal | grep 486 | grep re02
ioewe3h486hu5tnjdsre029y814mmq
nasa@nasa:~$ ./chal $(strings chal | grep 486 | grep re02)
Here is the flag: NASA2025{n4ndeharuh1ka93yatt4n0}
nasa@nasa:~$ _
```

Figure 37: 得到 flag

Flag of Problem 7.4:

NASA2025{n4ndeharuh1ka93yatt4n0}

7.5 tmux (4pt)

Reference:

- <https://blog.gtwang.org/linux/linux-tmux-terminal-multiplexer-tutorial/>
- <https://github.com/jbranchaud/til/blob/master/tmux/change-the-default-prefix-key.md>

- 首先，檢視 `.tmux.conf` 檔案，發現裡面將 `prefix` 改為 `Ctrl-a`。

```
unbind C-b
set-option -g prefix C-a
bind-key C-a send-prefix
```

Figure 38: Screenshot of `.tmux.conf`

- 我輸入 `tmux` 開啟一個新的 `tmux session`，並且開始切割畫面。
- 在 `tmux` 當中，將畫面切成左右兩塊的指令是 `prefix` 再輸入 `%`，將畫面切成上下兩塊的指令是 `prefix` 再輸入 `"`。以下為我的切分步驟：

```
1 Ctrl-a "  
2 Ctrl-a %  
3 Ctrl-a "  
4 Ctrl-a %  
5 Ctrl-a "  
6 Ctrl-a %
```

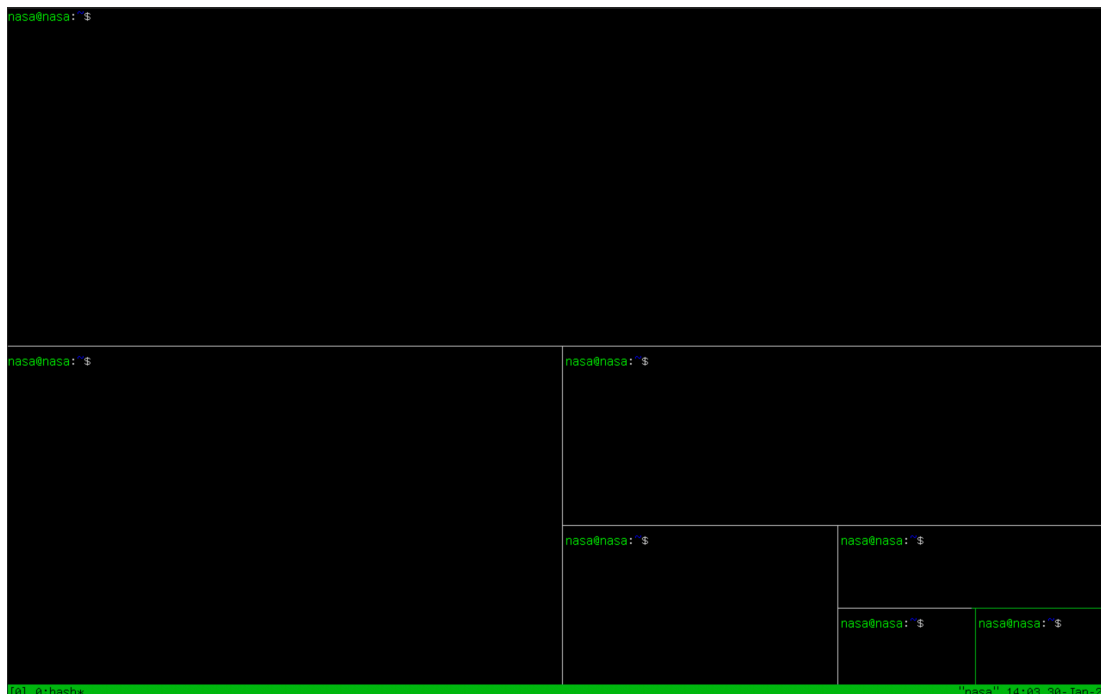


Figure 39: Screenshot of `tmux` panes

8. NASA 國的大危機 (10pt)

Reference:

- [Docker 基本教學 - iThome](#)
- [Dockerfile Overview - Docker 官方文件](#)
- [Docker 指令大全 - Pjchender](#)
- [Tcpdump 擷取封包指令範例教學 - Jinn's Blog](#)

Problem 8.1 解讀古老卷軸 (3pt)

- Dockerfile 內容如以下以及 Figure 40 所示

```
1 FROM python:3.9-slim
2 RUN apt-get update && apt-get install -y \
3     build-essential \
4     libssl-dev \
5     net-tools \
6     iproute2 \
7     tcpdump \
8     tshark \
9     nano \
10    curl \
11    wget \
12    vim \
13    less \
14    procs \
15    lsof \
16    iputils-ping \
17    && rm -rf /var/lib/apt/lists/*
18
19 RUN mkdir -p /usr/libexec/run
20
21 COPY usr/libexec/run/dist/transfer /usr/libexec/run/transfer
22 COPY usr/libexec/run/run.sh /usr/libexec/run/run.sh
23
24 RUN chmod +x /usr/libexec/run/transfer
25 RUN chmod +x /usr/libexec/run/run.sh
26
27 CMD ["/usr/libexec/run/run.sh"]
```

- 第 1 行：FROM python:3.9-slim 指令表示這個 Dockerfile 是基於 python:3.9-slim 映像建立的。這是一個精簡版的 Python 3.9 映像，基於 Debian，保留 Python 及必要的系統元件，但刪除了部分工具來減少映像大小。

- 第 2 - 16 行：RUN 代表在 Docker 映像建構（build）時執行的指令。這裡透過 apt-get 安裝了一些必要的 Linux 套件：
 - build-essential：編譯 C/C++ 必備的工具（如 gcc）。
 - libssl-dev：SSL 加密函式庫。
 - net-tools：網路工具（包含 ifconfig, netstat）。
 - iproute2：網路工具（包含 ip, ss）。
 - tcpdump、tshark：網路封包擷取與分析
 - nano、vim：文字編輯器。
 - curl、wget：HTTP 下載工具。
 - less：文字檢視工具，適合瀏覽長文件。
 - procs：包含 ps, top, kill 等指令，用於系統進程管理。
 - lsof：列出開啟的檔案，適合偵測哪些程序正在存取特定檔案或網路連線。
 - iputils-ping：提供 ping 指令，測試網路連線。
- 第 17 行：rm -rf /var/lib/apt/lists/* 這行指令清除 apt 快取，以減少 Docker 映像的大小。
- 第 19 行：mkdir -p /usr/libexec/run 確保 /usr/libexec/run 目錄存在，用於存放後續執行的腳本或工具。
- 第 21 - 22 行：COPY 指令將檔案從 Host 複製到容器內相對應的位置。
- 第 24 - 25 行：chmod +x 指令賦予 transfer 和 run.sh 執行權限：
- 第 27 行：CMD 可以指定容器啟動時的預設指令。當容器啟動時，會執行 run.sh 腳本。

```
FROM python:3.9-slim
RUN apt-get update && apt-get install -y \
    build-essential \
    libssl-dev \
    net-tools \
    iproute2 \
    tcpdump \
    tshark \
    nano \
    curl \
    wget \
    vim \
    less \
    procs \
    lsof \
    iputils-ping \
    && rm -rf /var/lib/apt/lists/*

RUN mkdir -p /usr/libexec/run

COPY usr/libexec/run/dist/transfer /usr/libexec/run/transfer
COPY usr/libexec/run/run.sh /usr/libexec/run/run.sh

RUN chmod +x /usr/libexec/run/transfer
RUN chmod +x /usr/libexec/run/run.sh

CMD ["/usr/libexec/run/run.sh"]
```

Figure 40: Dockerfile 內容截圖

Problem 8.2 啟動聖杯 (3pt)

- 首先我嘗試了使用指令直接啟動 docker 容器，但是發現無法成功啟動，如 Figure 41 所示。

```
nasa@nasa-hw0-pickle:~/mystic-cup/usr/libexec/run$ docker images
REPOSITORY      TAG         IMAGE ID      CREATED       SIZE
my-magic-cup    latest     3cddc4add21e  3 weeks ago  727MB
nasa@nasa-hw0-pickle:~/mystic-cup/usr/libexec/run$ docker run my-magic-cup
System routine check: FAILED. Exiting...
nasa@nasa-hw0-pickle:~/mystic-cup/usr/libexec/run$ _
```

Figure 41: 直接啟動 docker 容器

- 分析 Dockerfile 內容，可以知道在啟動容器時，會執行 /usr/libexec/run/run.sh 腳本。因此，我直接分析未被複製進去前的 run.sh 腳本，得知其內容如下：

```
#!/bin/bash

if [ "$MAGIC_SPELL" = "hahahaiLoveNASA" ]; then
    echo "System routine check: OK. Service started..."

    echo "Starting sending secret message..."
    /usr/libexec/run/transfer &
    tail -f /dev/null
else
    echo "System routine check: FAILED. Exiting..."
    exit 1
fi
```

Figure 42: run.sh 腳本內容

- 可以發現執行後，他會先檢查環境變數 MAGIC_SPELL 的內容是否為 hahahaiLoveNASA，若是則執行 transfer 檔案，否則印出 FAILED。
- 我使用以下指令在啟動 docker images 時，設定環境變數 MAGIC_SPELL 的值為 hahahaiLoveNASA，成功啟動容器。

```
1 docker run -e MAGIC_SPELL='hahahaiLoveNASA' my-magic-cup
```

```
nasa@nasa-hw0-pickle:~/mystic-cup$ docker run -e MAGIC_SPELL='hahahaiLoveNASA' my-magic-cup
System routine check: OK. Service started...
Starting sending secret message...
```

Figure 43: 成功啟動 docker 容器

Problem 8.3 神秘訊息 (4pt)

- 啟動 Docker container 並且 detach 之後，我使用指令 `docker ps -a` 查看開啟的 container

```
nasa@nasa-hw0-pickle:~/mystic-cup$ docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS        NAMES
ace4911d296d   my-magic-cup   "/usr/libexec/run/ru..." 9 minutes ago  Up 9 minutes                adoring_easley
nasa@nasa-hw0-pickle:~/mystic-cup$ _
```

Figure 44: Screenshot of `docker ps -a`

- 可以看到 container 的名稱為 `adoring_easley`，接著使用指令進入 container 內部

```
1 docker exec -it adoring_easley /bin/bash
```

- 在 docker 環境當中，我使用 `tcpdump` 指令來分析並且擷取封包。其中 `-i any` 參數表示監聽所有網路介面，`-A` 參數代表 Print each packet in ASCII，並成功找到 flag

```
root@ace4911d296d:/# tcpdump -i any -A
tcpdump: data link type LINUX_SLL2
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on any, link-type LINUX_SLL2 (Linux cooked v2), snapshot length 262144 bytes
07:24:59.766884 lo      In  IP localhost.5000 > localhost.6000: UDP, length 40
E..DT.@.@.....p.0.Cflag[I'll send our killer on 3948/02/22]
07:25:04.768658 lo      In  IP localhost.5000 > localhost.6000: UDP, length 40
E..DY(@.@..~.....p.0.Cflag[I'll send our killer on 3948/02/22]
07:25:09.770251 lo      In  IP localhost.5000 > localhost.6000: UDP, length 40
E..D].@.@.....p.0.Cflag[I'll send our killer on 3948/02/22]
07:25:14.771839 lo      In  IP localhost.5000 > localhost.6000: UDP, length 40
E..Da.@.@.....p.0.Cflag[I'll send our killer on 3948/02/22]
^C
4 packets captured
8 packets received by filter
0 packets dropped by kernel
root@ace4911d296d:/# _
```

Figure 45: Find the flag by using `tcpdump`

Flag of Problem 8.3:

```
flag[I'll send our killer on 3948/02/22]
```